Case Study

Credit Risk Analysis Using Classification and Regression Trees (CART)

German Credit (Statlog) Dataset

Student	Muhammad Naafir A
Programme	B.Tech Computer Science and Engineering, B.S. Abdur Rahman Crescent Institute of Science and Technology
Course component	Case Study (15 marks) - Modules III and IV
Dataset	Statlog German Credit Data (UCI / OpenML "credit-g"), 1,000 records, 20 attributes
Tools	Python 3, pandas, NumPy, SciPy, scikit-learn, matplotlib, seaborn (Google Colab)
Code and data	GitHub: https://github.com/naafir (notebook and frozen dataset CSV)
Date	11 November 2026

Abstract. This report applies the data preparation, re-expression, standardisation, outlier handling, partitioning and balancing techniques from Modules III and IV to the German Credit dataset, and then builds and evaluates a CART classifier (credit risk: good or bad) and a CART regressor (loan amount). The tuned classification tree reaches 65.3% accuracy, 0.567 recall for high-risk applicants and ROC-AUC 0.679; the regression tree reaches R^2 = 0.511 on the log of the loan amount.

1. Problem and Dataset

Banks must decide whether an applicant is likely to repay a loan. The Statlog German Credit dataset describes 1,000 past loan applicants with 20 attributes (checking account status, loan duration, credit history, purpose, credit amount, savings, employment, age, housing, job and others). Each applicant is labelled **good** (70%) or **bad** (30%) credit risk. It was chosen because it is a real-world dataset with a good mix of categorical and numeric fields, skewed distributions, a few outliers and an imbalanced target, so every technique in Modules III and IV has something to act on.

Two models are built from the same prepared data:

- **Classification tree:** predicts the credit risk class (target *Risk*: 0 = good, 1 = bad).
- **Regression tree:** predicts the loan amount (target: log of *credit_amount*).

2. Steps 1-2: Retrieving the Data and Data Preparation

The dataset was downloaded once from OpenML with `fetch_openml("credit-g")` and immediately saved as `credit_g_frozen.csv`. All later runs read this frozen file, so the data used in the presentation never changes. The data was inspected with `df.info()`, `describe()`, and missing-value and duplicate checks.

The following preparation changes were made:

Operation	Field	Change
Add index field	ID	New column ID = 1 to 1000, to identify each record
Change field values	class to Risk	Text labels good / bad converted to 0 / 1 (stored as integers)
Change field values	foreign_worker	yes / no converted to 1 / 0 (stored as integers)
Change field values	purpose	Labels tidied to title case (e.g. "new car" to "New Car")

The target and the binary field were deliberately stored as plain integers (not "category" type) so that they are not mistaken for categorical predictors in the next step.

3. Step 3: Identifying and Handling Categorical Data

Fields with text values were identified using `select_dtypes`. Twelve nominal categorical fields were found, and seven further fields are numeric (plus the binary *foreign_worker*).

Categorical field	Levels	Categorical field	Levels
checking_status	4	other_payment_plans	3
credit_history	5	housing	3
purpose	10	job	4
savings_status	5	own_telephone	2
employment	5	personal_status	4
other_parties	3	property_magnitude	4

These fields are nominal (their levels have no numeric meaning), so **one-hot (dummy) encoding** was applied with `drop_first=True` to avoid redundant columns. This changed the table from 22 columns to 50 columns (40 new dummy fields). Numeric codes such as 1, 2, 3 were not used because they would wrongly suggest an order between categories.

4. Step 4: Re-expression Techniques

The numeric fields *duration*, *credit_amount* and *age* were right-skewed (skewness above 1), which hurts distance-based methods and makes extreme values more influential. A **logarithmic re-expression**, log(1 + x), was applied. Skewness dropped to near zero for duration and credit amount, and to 0.43 for age.

Field	Skewness before	Skewness after (log)
duration	1.094	-0.051
credit_amount	1.950	0.130
age	1.021	0.432

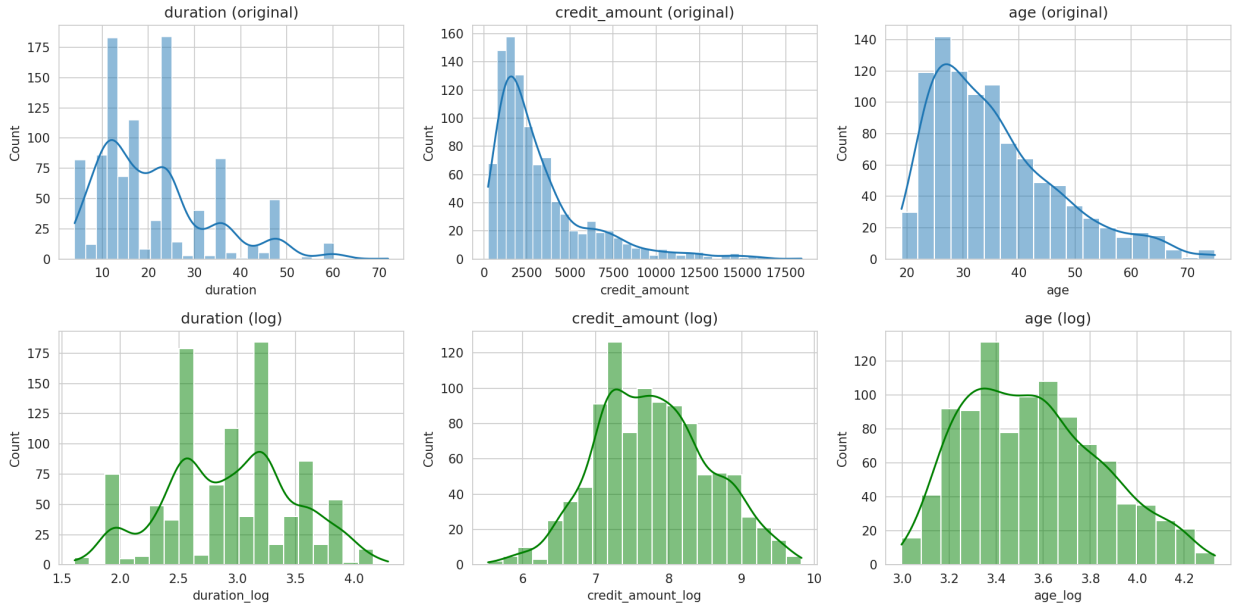


Figure 1. Distributions before (top) and after (bottom) the log re-expression.

5. Step 5: Standardising the Numeric Data

The seven numeric predictors (the three log-transformed fields, *installment_commitment*, *residence_since*, *existing_credits* and *num_dependents*) were converted to **z-scores**, z = (x - mean) / standard deviation, using `StandardScaler`. The result has mean 0.000 and standard deviation 1.001 (scikit-learn uses the population formula; pandas reports the sample formula, hence 1.001). The standardised fields carry the suffix *_z*. Standardising puts all numeric fields on one scale, makes values comparable across units, and allows outliers to be identified with the |z| > 3 rule.

6. Step 6: Identifying and Handling Outliers

Outliers were identified in two ways: the |z| > 3 rule and boxplots. The z-score rule flagged **6 values**, all in *existing_credits* (applicants with an unusually high number of existing credits); no other field had values beyond 3 standard deviations.

Field (z-scored)	Outliers with |z| > 3
duration_log_z	0
credit_amount_log_z	0
age_log_z	0
installment_commitment_z	0
residence_since_z	0
existing_credits_z	6
num_dependents_z	0

Because the dataset has only 1,000 records, deleting rows would waste information. The six values were instead **capped (winsorised) at +/-3**. After capping, no value exceeded |z| = 3. The boxplots also show a few mild points beyond the whiskers (for example in duration, credit amount and number of dependents); these lie within +/-3 standard deviations, are genuine values, and were kept.

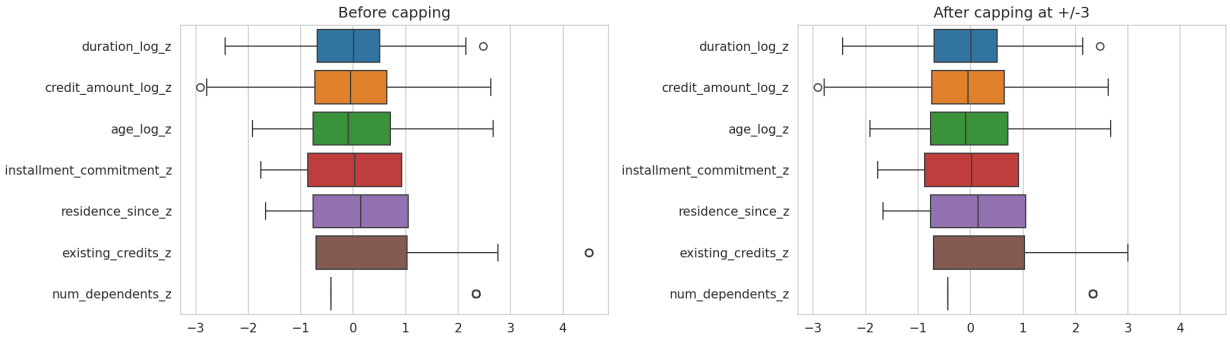


Figure 2. Boxplots of the standardised fields before (left) and after (right) capping at +/-3.

7. Step 7: Partitioning into Training and Testing Sets

The data was split 70% / 30% with `train_test_split` (*random_state* = *42*, **stratified** on Risk so that both parts keep the same good/bad ratio). The 48 predictors are the 7 standardised numeric fields, the 40 dummy fields and *foreign_worker*. The ID and the target are not used as predictors.

Training set: **700 records** x 48 predictors. Testing set: **300 records** x 48 predictors.

8. Step 8: Validating the Partition and Balancing the Training Data

Validation. The target proportions are identical in both parts (70% good, 30% bad), and a chi-square test on the proportions gives p = 1.000 (as expected from stratification). Welch two-sample t-tests comparing the numeric fields between training and testing sets all give p > 0.05, so there is no evidence that the two partitions differ.

Field	Train mean	Test mean	p-value
duration_log_z	-0.012	0.028	0.555
credit_amount_log_z	-0.017	0.041	0.406
age_log_z	0.003	-0.006	0.898
installment_commitment_z	-0.031	0.072	0.135
residence_since_z	-0.023	0.053	0.278
existing_credits_z	-0.014	0.002	0.824
num_dependents_z	0.010	-0.023	0.630

Balancing. The training set is imbalanced (490 good vs 210 bad). A tree trained on it tends to favour the majority class, so the minority class (bad) was **randomly oversampled** with replacement to 490 records, giving 490 good and 490 bad. **Only the training set was balanced**; the testing set was left untouched so that the evaluation reflects the real class mix.

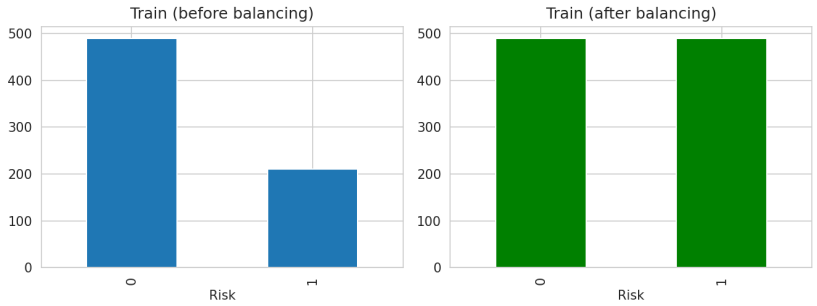


Figure 3. Class counts in the training set before and after balancing (0 = good, 1 = bad).

9. Step 9: Building the CART Classification Model

A `DecisionTreeClassifier` (the CART algorithm: binary splits chosen to minimise impurity) was trained on the balanced training set. Hyper-parameters were tuned with 5-fold cross-validated grid search (scoring: F1) over *criterion* (gini, entropy), *max_depth* (3, 4, 5, 6, 8) and *min_samples_leaf* (5, 10, 20). Limiting depth and leaf size controls over-fitting.

Best parameters: criterion = gini, max_depth = 6, min_samples_leaf = 10.

The first split of the tree is on *checking_status_no checking*. Applicants who do have a checking account (left branch, 655 training records: 246 good, 409 bad) are mostly predicted **bad**, whereas applicants with no checking account (right branch, 325 records: 244 good, 81 bad) are mostly predicted **good**. The tree on the next page shows the top three levels.

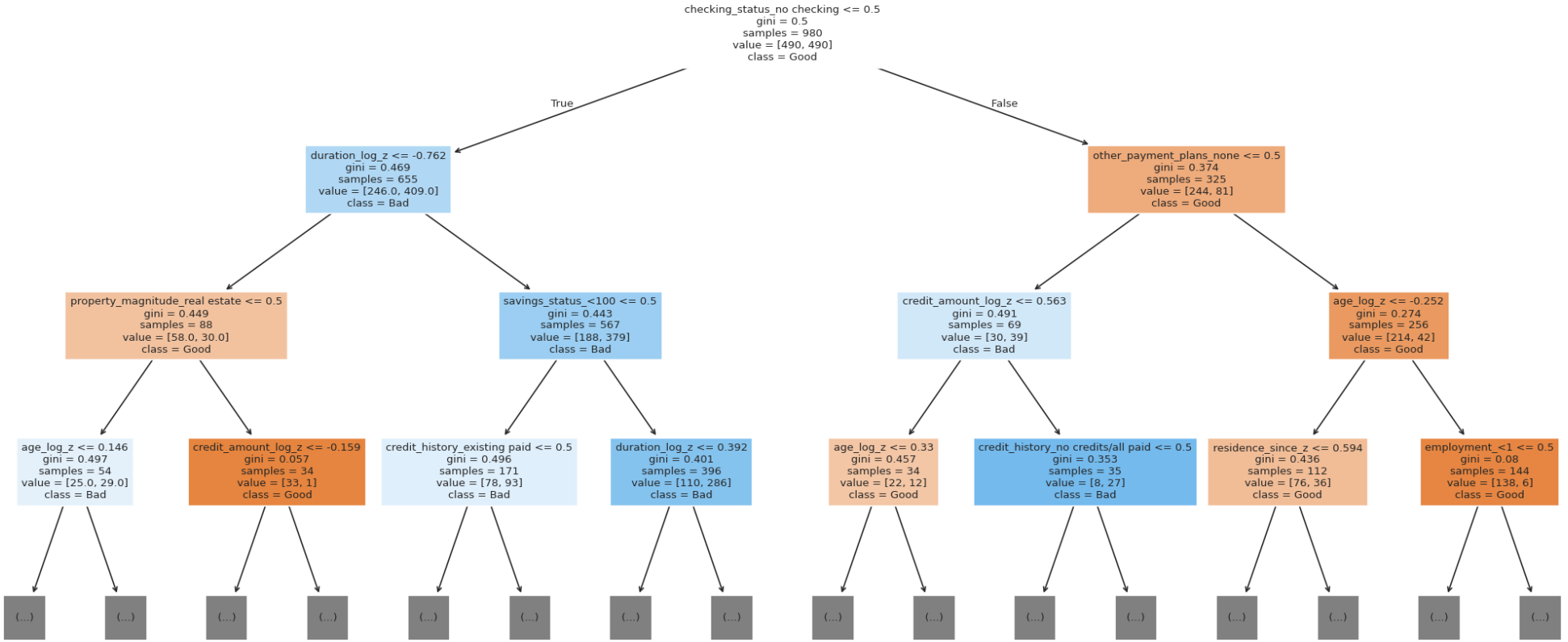


Figure 4. Top three levels of the fitted CART tree (orange = predicts good, blue = predicts bad; darker = purer node).

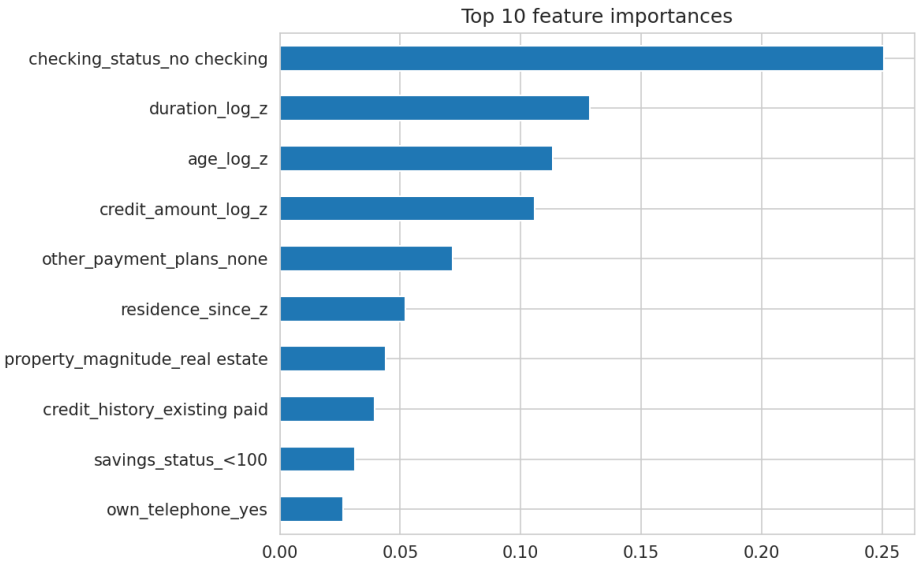


Figure 5. Ten most important features of the classification tree.

The most important predictor is **checking_status_no checking** (importance about 0.25), followed by loan **duration** (about 0.13), **age** (about 0.11), **credit amount** (about 0.105) and **other_payment_plans_none** (about 0.07). This agrees with domain intuition: the state of the applicant's bank account, the length and size of the loan, and other credit commitments are the main signals of repayment risk. Feature importance measures how much a field reduces impurity in the tree; it does not by itself prove a causal effect.

10. Step 10: Model Evaluation

The tuned tree was evaluated on the 300 unseen test records. "Bad" credit risk is treated as the positive class.

Metric	Value	Meaning
Accuracy	0.653	Share of all applicants classified correctly (196 of 300)
Precision (bad)	0.440	Of applicants flagged as bad, 44% really are bad (51 of 116)
Recall (bad)	0.567	Of truly bad applicants, 56.7% were caught (51 of 90)
F1-score	0.495	Harmonic mean of precision and recall
ROC-AUC	0.679	Ability to rank bad applicants above good ones (0.5 = random)

Confusion matrix	Predicted good	Predicted bad
Actually good (210)	145 (true negatives)	65 (false positives)
Actually bad (90)	39 (false negatives)	51 (true positives)

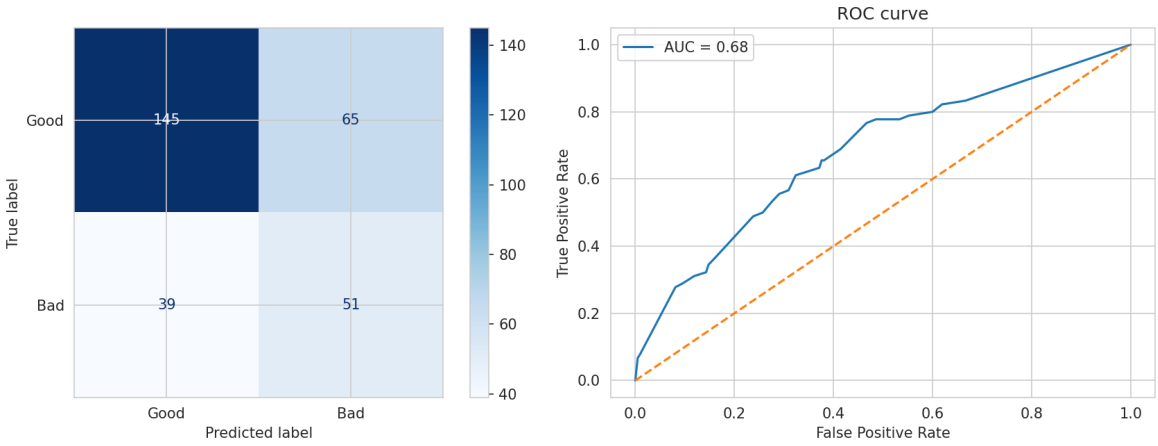


Figure 6. Confusion matrix (left) and ROC curve (right) on the test set.

Effect of balancing. The same tuned tree trained on the unbalanced training set caught only 46.7% of bad applicants (recall 0.467). After balancing, recall rose to 0.567, an improvement of 10 percentage points, because the tree no longer favours the majority class. The price is more false alarms (65 good applicants flagged as bad), which lowers precision and accuracy.

Interpretation. Always predicting "good" would score 70% accuracy on this test set but would catch no risky borrower at all, so accuracy alone is misleading here. In lending, approving a borrower who defaults is normally costlier than rejecting a good one, which is why recall for the bad class was prioritised. An AUC of 0.68 is modest but typical for a single, shallow decision tree on this small and noisy dataset, and clearly better than random guessing (0.5).

11. CART Regression Tree: Predicting the Loan Amount

To use CART for a numeric target, a `DecisionTreeRegressor` was trained to predict the log of the credit amount (the log scale was used because the raw amount is strongly skewed). The credit-amount feature itself was removed from the predictors to avoid leakage. Grid search (5-fold, mean squared error) over *max_depth* and *min_samples_leaf* selected **max_depth = 5, min_samples_leaf = 10**.

Metric (test set)	Value
R^2	0.511
MAE (log scale)	0.417
RMSE (log scale)	0.552

R^2 = 0.511 means the tree explains about half of the variation in loan amount. An MAE of 0.417 on the log scale means a typical prediction is off by a factor of about 1.5 ($e^{0.417}$ = 1.52). The scatter plot shows the staircase pattern typical of trees (each leaf predicts one constant value) and a tendency to under-predict the largest loans.



Figure 7. Regression tree: actual vs predicted log(loan amount); the dashed line is perfect prediction.

12. Conclusion

The German Credit data (1,000 applicants, 30% high risk) was prepared by adding an ID field, recoding the target and the binary field, one-hot encoding 12 categorical fields, log-transforming the skewed numeric fields, standardising them to z-scores and capping six outliers. A stratified 70/30 split was validated with chi-square and t-tests (all $p > 0.05$) and the training set was balanced by oversampling. The tuned CART classifier (Gini, depth 6, minimum leaf size 10) achieved 65.3% accuracy, 0.567 recall for high-risk applicants and ROC-AUC 0.679. Balancing raised recall from 0.467 to 0.567 at the cost of precision (0.44), a reasonable trade-off because missing a risky borrower is costlier than rejecting a good one. The strongest predictors were the absence of a checking account, loan duration, age and credit amount. The regression tree for loan amount reached $R^2 = 0.511$.

Limitations

- A single decision tree has high variance and the dataset is small, so results may change with a different split.
- Oversampling duplicates minority records, which can make the tree slightly over-confident on the training data.
- Only one train/test split was used; the test set has just 90 high-risk cases, so the metrics carry noticeable uncertainty.

Future work

- Cost-complexity pruning and repeated cross-validation to obtain more stable estimates.
- Ensembles such as Random Forest or gradient boosting, and SMOTE or class weights as alternatives to plain oversampling.
- Choosing the decision threshold from the real cost of a bad loan versus a rejected good loan.

Reproducibility

All steps are in the Google Colab / GitHub notebook (https://github.com/naafir). The dataset is frozen in `credit_g_frozen.csv` and every random operation uses *random_state = 42*, so running the notebook from top to bottom reproduces every number in this report.